SahAI — AI Voice Co-Pilot for Inside Sales

Complete project brief · everything the team needs before the presentation

Track	AI Build 2026 · Voice Co-Pilot (Pay-in-3)
Repository	`github.com/KL2300030695/SahAI`
Stack	Groq (5 open-weights models) · FastAPI · React 18 · Chroma + BM25 · SQLite · Firestore · Google Sheets · Docker
Scale	64 source files · ~11,100 lines · 187 tests, all offline
Measured cost	**$0.0037 per call (₹0.31)** — 55× cheaper than a single frontier mega-prompt

6	8	28%	187
AGENTS	GUARDRAILS (6 IN CODE)	STAGES AT ZERO COST	TESTS PASSING

Contents

1 · The one-paragraph version

SahAI listens to a live sales call and puts **one sentence** on the agent's screen — the next thing to say — with every figure in it underlined and traced back to the exact clause of the product handbook it came from. It understands intent, retrieves the right terms, checks its own output against eight guardrails, and after the call writes a summary, proposes a CRM update and drafts a follow-up. It never speaks to the customer, and it cannot write to a customer record on its own. Both of those are enforced by code and a state machine, not requested in a prompt.

2 · The problem, and why it is hard

A fintech has launched **Pay-in-3**: a purchase split into three equal monthly instalments at zero additional cost. The product is genuinely good — the customer repays exactly the cart value. It is also *hard to sell honestly*, for three reasons:

- **"Zero-cost" invites disbelief.** The honest answer involves a merchant-funded business model, plus a late fee and a bounce fee that *do* exist. An agent who oversimplifies creates a complaint later.
- **KYC is where people leave.** Aadhaar and PAN steps trigger privacy anxiety, and the recovery script differs per objection.
- **Consistency, not capability, is the gap.** The best agent already quotes the right fee and logs the real drop-off reason. The other nineteen do it unevenly. The lever is **variance**.

3 · What the product actually is

> **Co-pilot, not bot**
> The AI never addresses the customer. A human reads the suggested line and says it in their own words. That is what makes the guardrails meaningful — there is always a person between the model and the customer, and that person can disagree.

The agent sees a single serif sentence at speaking size. Everything else on screen exists to answer one question: *can I trust that line?* Figures traced to a source are underlined inside the sentence itself, not listed in a footnote panel — because an agent mid-call has under a second to look.

4 · The pipeline, stage by stage

Every stage has a typed input and output. Cost and tier are recorded for each one.

```
 AUDIO IN  — microphone (browser VAD) — or — uploaded recording — or — scripted replay
     │
     ▼
[0] CONSENT GATE          code        raises if consent is not on record — no bypass path exists
     │
[1] TRANSCRIBE            STT         whisper-large-v3-turbo, one utterance at a time
     │                                TranscriptTurn{index, speaker, text, ts}
[2] INJECTION SCREEN      TINY        llama-prompt-guard-2-86m · an attack never reaches a
     │                                reasoning model — halted for ~$0.000001
[3] INTENT                CHEAP       llama-3.1-8b-instant
     │                                {intent, confidence, entities, sentiment, dropoff_risk}
[4] RETRIEVAL             NONE        Chroma (local ONNX MiniLM) + BM25 → reciprocal rank fusion
     │                                $0.00 · single-digit ms · 83 chunks from 19 documents
[5] NEXT BEST ACTION      STANDARD    gpt-oss-20b, escalating to gpt-oss-120b by CODE rules
     │  → HIGH                       {say, why, action_type, cited_chunk_ids, requires_human_confirmation}
[6] SELF-CHECK            code →      8 checks; 6 are plain Python. Then gpt-oss-safeguard-20b
     │  SAFETY/HIGH                  adjudicates conduct against the Business Goals
     ▼
  SURFACED TO THE AGENT   WebSocket push — event-driven, not polled
     │
  ═══ end of call ═══
     ▼
[7] POST-CALL             HIGH        summary · disposition · drop-off reason · CRM patch ·
     │                                follow-up draft · conversion probability
[8] SELF-CHECK (post)     code+HIGH   internal note vs customer-facing message are judged differently
     ▼

╔═══════════════════════════════════════════════════════════════════════════╗
  AUTOMATION STOPS.  send_status = pending_agent_approval
╚═══════════════════════════════════════════════════════════════════════════╝

     ▼
[9] HUMAN APPROVAL        —           the only path that writes to a customer record
```

Agents never import each other

Every agent's input and output type lives in `app/schemas.py`, and agents import from there and nowhere else. That single constraint is what makes this a multi-agent system rather than a mega-prompt with headings: each agent can be tested, swapped or re-tiered in isolation because its only coupling is to a Pydantic type. The orchestrator is the one module that knows the shape.

Routing is code, not prompt text

Six named predicates decide whether a turn goes to the 20B or the 120B model. They are served at `/api/policy` so the tiering is inspectable rather than a claim on a slide.

Rule	Fires when
`sensitive_intent`	intent is eligibility, objection_cost, objection_trust, complaint or payment_issue
`credit_terms_in_context`	the *customer's own words* mention credit terms
`high_dropoff_risk`	risk > 0.6
`low_intent_confidence`	classifier confidence < 0.6
`negative_sentiment`	frustrated, angry, or in a hurry
`agent_requested`	the agent asked for a second opinion

A costly bug worth telling

The `credit_terms_in_context` rule originally read the *retrieved knowledge-base text* as well as the customer's words. That sounds reasonable and is badly wrong: this is a credit product, so every retrieved chunk mentions fees — and roughly 90% of turns escalated to the expensive model. Scoping it to the customer's own words took cost from $0.0043 to $0.0032 per call with no loss of safety.

Retrieval refuses to guess

Reciprocal rank fusion scores *rank position*, not relevance. Before a confidence floor existed, the top hit for "*how do I reset my wifi router*" scored exactly as highly as the top hit for "*what is the late fee*" — four confident-looking chunks with real IDs came back for both. The floor was measured, not guessed:

Cosine similarity on the intent-expanded query	min	max
in-domain (20 utterances)	**0.4807**	0.7008
off-topic (12 utterances)	0.1406	**0.4480**

`MIN_COSINE = 0.40`. Below it **no citations are returned at all** — not a low-confidence flag beside the chunks. A model handed plausible fee tables will quote them whatever flag sits alongside; withholding the text is the only thing that works. BM25 stays a *ranking* contributor (it catches "₹250" and "PAN") and is never consulted for confidence, because measured on this corpus the wifi-router query outscores most real product questions on BM25 alone — it rewards the common tokens in "how do I". Reproduce with `python -m scripts.calibrate_retrieval`.

5 · Cost: how 55× is achieved

Every figure comes from the `usage` field of real API responses, priced by `backend/app/config.py`, and persisted per decision. Nothing is estimated.

Seed scenario	Stages	Tokens	SahAI	One frontier mega-prompt	Ratio
call-001 · won / hidden charges	47	33,024	$0.004134	$0.2356	**57×**
call-002 · dropped / Aadhaar	48	31,527	$0.004592	$0.2191	**48×**
call-003 · won / credit score	54	34,359	$0.004270	$0.2423	**57×**
call-004 · not interested	31	16,101	$0.001860	$0.1126	**61×**
Mean			**$0.003714** ₹0.31	**$0.2024**	**55×**

At **10,000 calls/month: $37 vs $2,024.** At 100,000: $371 vs $20,240.

Where the money goes

Tier	Model	Stages	Share of cost
high	`openai/gpt-oss-120b`	59	42.2%
standard	`openai/gpt-oss-20b`	53	21.3%
stt	`whisper-large-v3-turbo`	92	13.8%
safety	`openai/gpt-oss-safeguard-20b`	48	13.5%
cheap	`llama-3.1-8b-instant`	87	9.0%
tiny	`llama-prompt-guard-2-86m`	88	0.1%
none	**local compute**	**185**	**0.0%**

> **185 of 652 pipeline stages (28%) cost nothing at all**
> Retrieval and six of the eight guardrails are local compute. That is the concrete form of the brief's principle that "a smaller open-source model, RAG, or a classical solver could replace constant calls to an expensive commercial LLM".

THREE LEVERS, NOT ONE

1. **Cost tiering.** An 86M classifier screens every turn; an 8B model does intent; a 20B model handles routine suggestions; the 120B is reserved for credit terms and objections.
2. **RAG instead of inference.** Retrieval is the highest-frequency step and costs $0.00 — local embeddings plus BM25, no torch.
3. **Reasoning-effort control.** The reasoning models bill chain-of-thought as output tokens. Measured on an identical prompt: `low` = 150 completion tokens, `medium` = 334 — 2.2× the cost for the same answer.

The baseline is deliberately conservative: it prices only the tokens actually spent. A real mega-prompt would also carry the whole handbook in context on every turn instead of retrieving four chunks, so the true gap is wider than the number claims.

6 · The eight guardrails

`enforced_by` is shipped to the UI so a reviewer can see which checks are un-promptable code and which are model judgement. We do not blur the distinction.

Check	By	What it catches
`consent_recorded`	CODE	Any turn processed without consent on record
`pii_redaction`	CODE	Aadhaar, PAN, card, phone — masked before storage *and* display
`grounding`	CODE	Any figure not present verbatim in a cited chunk
`no_stale_terms`	CODE	Clauses past their `effective_to` date
`no_autonomous_credit_terms`	CODE	Forces human confirmation on credit language
`no_fabricated_actions`	CODE	"I've already sent you an email" — the worst available failure
`injection_screen`	LLM	Prompt-injection attempts
`goal_alignment`	LLM	Drift from the business conduct policy

Grounding asks a different question after the call

A live suggestion is grounded against the **handbook**. A post-call summary is grounded against the **call it summarises** — because a summary's figures come from the conversation, not the knowledge base. Asking the handbook question of a summary was a guaranteed false positive (post-call retrieval passes no citations), so every summary mentioning any number failed. What the transcript check catches is real: a summariser inventing a number and writing it into a customer's record.

Grounding and the interface share one function

`ground_figures()` returns the character offsets the Say Line underlines *and* the verdict the guardrail blocks on. One function, one source of truth — the marks the agent sees and the decision to block can never disagree.

7 · Where automation stops

```
AUTOMATIC — no human in the loop
├─ transcription → intent → retrieval → suggestion → surfaced to the agent
├─ post-call summary, disposition, drop-off reason
├─ CRM patch          ← PROPOSED, written to the call log only
└─ follow-up message  ← DRAFTED

═══════════════ send_status = pending_agent_approval ═══════════════

REQUIRES A NAMED, AUTHENTICATED HUMAN
├─ applying the patch to the customer record
└─ sending the message to the customer
```

`POST /api/calls/{id}/approve` is the only route that can move that state. It requires a capability the credential carries, and **no agent can call it**.

The failure this caught, in our own system

The post-call check correctly caught a draft claiming Pay-in-3 was entirely free with no mention of the late fee, and wrote the failing verdict to the trace. The approve endpoint *never read it* — the message was written and marked `sent`, with the check that rejected it rendered on the same screen. **A guardrail nothing consumes is decoration.** The verdict is now load-bearing: a blocked message is refused, and the way past it is a rewrite, not another click. An edited message is re-checked against the deterministic rules, and the override is recorded in the audit trace under the approver's name — because the accountable act is a human choosing different words, not a human dismissing a warning.

8 · Enterprise architecture

Access control, and who signs an approval

> **Another claim that was unenforced**
> The approver's name used to arrive as a *string in the request body*, and nothing was authenticated. Anyone who could reach the port could approve as anyone. A human-oversight gate whose identity is self-asserted is a convention, not a control.

The approver is now who the credential says they are. The request body no longer carries a name at all, and the dashboard shows the identity it will sign with rather than asking for one — a field the server ignores teaches the wrong thing about where authority comes from.

Role	read	run calls	approve	sync integrations
`viewer`	✓			
`agent`	✓	✓	✓	
`admin`	✓	✓	✓	✓

- Keys compared with `hmac.compare_digest` — no timing side channel.
- Reads stay open, so a fresh clone still shows a working dashboard. Writes never do.
- A write made while auth is off is stamped `(unauthenticated)` in the audit trail — otherwise, a year later, nobody could tell a signed approval from one made on an open port.
- **Sign-in exists; sign-up deliberately does not.** Nobody self-registers for the system that writes to customer records. The screen states the provisioning model where a "create an account" link would sit.

The CRM is a port, not a table

The pipeline talks to a four-method interface. Nothing outside that module knows what is behind it.

```
read_snapshot  ·  is_do_not_call  ·  apply_patch  ·  describe
```

> **Why `is_do_not_call` is separate from the snapshot**
> Both read the same record, but a snapshot is *advisory context for a language model* and do-not-call is a *legal obligation checked in code before drafting*. Merging them would make the obligation depend on a field surviving a model's attention.

Two adapters ship. `SqliteCrm` is the demo default. `RestCrm` talks to any HTTP CRM and is **configured, not coded** — field names included, because `kyc_status` here is `KYC_Status__c` there. Its failure policies differ by method on purpose, and each has a test:

Failure	Policy	Why
read fails	degrade to no snapshot	less context is worse assistance, not a safety problem — the call continues
do-not-call check fails	**return True**	an unreachable CRM is not permission to call someone who may have opted out
write fails	report, leave pending	nothing is silently lost

`RestCrm` is tested against a real stub HTTP server, not a mock — so the claim is "this works against an HTTP CRM", not "we imagined one".

Deployment and operations

```
docker compose up --build      →   http://localhost:8000
```

One service, not three: the dashboard is built into the same image that serves the API, so there is no proxy to configure and no CORS to get wrong between a laptop and a server. The knowledge base is indexed at *image build time* — otherwise the first call of a demo pays to embed 19 documents and a cold container looks broken rather than slow.

Endpoint	Purpose
`GET /healthz`	Liveness. Checks *nothing else* — a probe that touches the database restarts a healthy container into a crash loop when the database blips.
`GET /readyz`	Readiness: database + knowledge-base index. **Not** the model provider — a quota failure is already a per-request 503 with an explanation, and pulling the instance would take the dashboard down too.
`every response`	`X-Request-ID` and `X-Response-Time-ms`. A call fans out across six agents and two mirrors; timestamps stop correlating the moment two agents are on the phone at once.

Where the data goes

Destination	Role	Status
SQLite	System of record — transactional, sub-millisecond, offline	LIVE
Firestore	Durable cloud mirror · `sahai_calls` + `stages` subcollections	38 CALLS, 671 DOCS
Google Sheets	Calls tab + Trace tab, upserted by `call_id`	38 CALLS, 632 STAGES
CSV	`/api/export/calls.csv` and `/api/export/trace.csv`	LIVE

> **Why SQLite stays the system of record**
>
> `get_crm_snapshot` runs on *every turn* inside a ~1s latency budget. A Firestore read from India is a 50–200 ms round trip — a fifth of the budget spent on network, and a demo that fails whenever the wifi does. Every write lands locally first and is mirrored afterwards. All the data reaches Firestore; the customer on the phone never waits for it. Both mirrors are best-effort: an outage is counted and reported at `/api/integrations/status`, never raised. There is a test that an approval still succeeds with both mirrors throwing.

One row builder feeds all four destinations. The mirrors consume the same functions the CSV endpoints use, so a file, a Firestore document and a spreadsheet row can never disagree about what a call cost.

9 · The dashboard, screen by screen

SIGN IN

One screen. Your key decides what you can do and is the name that goes on anything you approve. No self-registration — the screen says accounts are provisioned by an administrator.

IDLE

The opening consent script is shown *before* the call starts, so the agent reads it rather than improvising. Below it: start a microphone call, or rehearse on one of four past calls.

CONSENT

Full-width, serif, at speaking size. Nothing runs until it is on record — the co-pilot does not warn about missing consent, it refuses to process a single turn without it.

LIVE CALL

Element	What it does
Say Line	The signature element. One sentence, serif, 23px — the only large type on screen. Traced figures underlined *inside* the sentence. Four states: ready · you confirm this · held · listening.
Conversation	Customer turns carry more weight than the agent's own — the agent already knows what they said. Intent chips per turn.
Their record	The CRM, with a dot marking exactly the fields the co-pilot reads. Do-not-call renders first and loud. Credit limit is shown to the agent but deliberately *not* sent to the model.
What this used	The retrieved chunks, with document title, version and chunk ID.
Evidence strip	Collapsed by default. Every check, how it was enforced, the running cost and the ratio.

APPROVAL

The mirror of consent. The call opened with words a human spoke; it closes with a decision a human signs. CRM changes render as a real before → after diff with strikethrough. "Nothing has been written yet" until you act.

Design decisions worth defending

- **Inline provenance over a citations panel.** An agent mid-call has under a second. A footnote list is read after the fact, or never.
- **The microphone goes deaf while the co-pilot speaks.** On a laptop the suggestion comes out of the speakers next to the mic; without a gate the pipeline transcribes its own voice, files it as the customer, and answers itself. Nothing in the transcript looks wrong — it just fills with fluent sentences nobody said.
- **The evidence strip reports findings, not consequences.** It once read "N checks stopped this" above a call that had been signed off and written. The Say Line and the approval band report consequences; they are the two places that know.

10 · Data and the knowledge base

Source the brief asks for	How we use it
Call recordings / transcripts	Whisper on live mic and uploads; four seeded transcripts for rehearsal
CRM records and past interactions	`past_interactions` and `last_disposition` reach the suggestion prompt; shown to the agent in "Their record"
Product, offer, KYC process information	19 documents → 83 chunks, versioned with validity windows
Successful vs unsuccessful conversations	Two won and two dropped seed calls, with the drop-off reason captured per call

Stale terms are exercised, not assumed

The knowledge base deliberately contains an **expired 2024 fee schedule**. Three chunks are past their `effective_to` date. They are dropped at retrieval — before the reasoning model sees them — and the `no_stale_terms` guardrail is the second line of defence. This is how the brief's "never quote stale terms" rule is proven rather than claimed.

11 · Running it

PREREQUISITES

Python 3.10+, Node 18+, a Groq API key.

```
# 1 · backend
cd backend
python -m venv .venv
.venv\Scripts\activate           # Windows      (source .venv/bin/activate on macOS/Linux)
pip install -r requirements.txt

# 2 · .env in the REPO ROOT (gitignored — never commit it)
GROQ_API_KEY=gsk_your_key_here
MODEL_TINY=meta-llama/llama-prompt-guard-2-86m
MODEL_CHEAP=llama-3.1-8b-instant
MODEL_STANDARD=openai/gpt-oss-20b
MODEL_HIGH=openai/gpt-oss-120b
MODEL_SAFETY=openai/gpt-oss-safeguard-20b
MODEL_STT=whisper-large-v3-turbo
SAHAI_MOCK=0                     # 1 = run with no API calls at all
AUTH_ENABLED=1
API_KEYS=k_agent_priya:Priya Nair:agent,k_view_anita:Anita Rao:viewer,k_admin_ravi:Ravi Menon:admin

# 3 · build the knowledge base and seed the CRM
python -m app.rag.ingest         # 19 documents → 83 chunks
python -m app.seed.seed_db

# 4 · run  (two terminals)
uvicorn app.main:app --reload --port 8000
cd frontend && npm install && npm run dev      →  http://localhost:5173
```

OR ONE COMMAND

```
docker compose up --build        →  http://localhost:8000
```

USEFUL COMMANDS

```
pytest                                          # 187 tests, no network
python -m scripts.run_full_pipeline call-001     # one call through every stage + cost breakdown
python -m scripts.calibrate_retrieval            # reproduce the retrieval floor from measurements
curl localhost:8000/api/policy                   # the routing policy and pricing, as served
curl localhost:8000/readyz                       # database, index, auth mode, CRM connector
curl -X POST localhost:8000/api/integrations/sync -H "X-API-Key: k_admin_ravi"
```

> **Groq's free tier is ~7 full calls per day**
>
> 200,000 tokens/day, and the cap is **per organization, not per API key** — issuing a new key on the same account does not reset it. Do not rehearse on the same UTC day as the demo. `SAHAI_MOCK=1` runs every path with nothing billed.

12 · Scoring: how we meet the rubric

Dimension	Wt	Where it lands
Business Impact	20%	The lever is variance, not capability. Every agent quotes the right fee, pre-empts the Aadhaar step, logs the real drop-off reason, and no drop-off goes un-followed-up. Unit economics measured, not modelled.
AI Innovation & Depth	20%	Six cooperating agents coupled only to schemas. Routing by code predicates. Hybrid RAG with a *measured* confidence floor. Grounding that returns character offsets so the UI and the verdict share one function.
Technical Excellence	20%	187 offline tests written against failures that actually happened. Deterministic guardrails. Provider failures translated into sentences, not 500s. Contract tests against a real stub server.
Enterprise Architecture	15%	CRM as a four-method port with two adapters. API-key auth with roles, identity from the credential. Docker one-command deploy, liveness/readiness split, request correlation. Four output destinations from one row builder.
User Experience	10%	One sentence at speaking size with inline provenance. Two weighted human moments. Interface says what it wants from you in plain language, never a raw status code.
Scalability, Security & Cost	10%	$0.0037/call measured. 28% of stages at zero marginal cost. PII masked before storage and display. Consent as a code gate. Auth enforced. Secrets gitignored and excluded from the build context.
Presentation	5%	README, business pitch, this brief, and a rehearsed 8-minute script.

13 · Known limits, stated

Naming your own gaps reads far better than being caught by them. Every one of these is in the README under "Known limits".

Limit	Detail
Grounding validates only numeric claims	Every figure is verified against its source, but a sentence like *"we never share your Aadhaar"* contains no numbers and passes unchecked. Only the `goal_alignment` LLM check sits beneath it. **This is the largest open gap** and we say so first.
Two-party speaker attribution	One microphone cannot separate two speakers, so every utterance is attributed to the customer. Assumes speakerphone or a solo run. Real diarization needs a carrier integration or a diarizing STT model.
Telephony was removed deliberately	An earlier version took real inbound calls over Twilio and got exact attribution from dual tracks. Cut because a phone number is a recurring cost. The orchestrator never knew about it, so restoring it is one adapter, not a redesign.
Live-call state is in memory	A backend restart loses in-flight calls. Fine for one process; a real deployment needs Redis.
No multi-tenancy	Principals carry a role but not an organisation, so data is not tenant-scoped. The `Principal` type is the seam where it would go.
Groq quota	~7 full calls per free-tier day, per organization.

14 · Bugs we found and fixed

Useful to know — a judge may probe any of these, and the honest answer is stronger than a deflection.

Bug	Why it mattered
Guardrail verdict was never read at the approve gate	A message the system itself rejected was written and marked `sent`. The core safety claim was false at the one place it had to hold.
Approver identity was a request-body string	Anyone reaching the port could approve as anyone. Human oversight was a convention, not a control.
Routing read the retrieved KB text	Every chunk mentions fees in a credit product → ~90% escalation. Cost fell 25% when scoped to the customer's words.
Retrieval had no confidence floor	RRF scores rank, not relevance — off-topic queries returned four confident-looking citations.
Post-call grounding used the wrong source	Every summary containing any number failed. A guardrail that cries wolf teaches agents to ignore the panel where real blocks appear.
PII redaction ordering	A 16-digit card's first 12 digits matched the Aadhaar pattern → `card [AADHAAR_REDACTED] 1111`, leaking four digits.
The co-pilot's TTS fed back into the microphone	The pipeline transcribed its own voice as the customer and answered itself.
The test suite wrote to the live Firestore project	187 tests passed while creating documents named `test-approval-gate` beside real calls.
Sheets bulk sync hit Google's read quota	Stopped at 19 of 38 and left a half-filled tab that looks exactly like missing data.
Quota exhaustion surfaced as "500 Internal Server Error"	Told an agent with a customer on the line nothing about whose fault it was.

15 · Demo script — 8 minutes

Time	Beat	Say / show
0:00	Problem	"Pay-in-3 is genuinely zero-cost — and that is exactly why it is hard to sell. 'Zero cost' invites disbelief, the honest answer involves a late fee, and twenty agents answer twenty different ways."
0:45	Sign in	Sign in as Priya · agent. "Your key decides what you can do — and it is the name that goes on anything you approve."
1:15	Consent	Show the script pre-written. "Nothing runs until this is on record. Not a warning — the orchestrator raises."
1:45	**The Say Line**	**The core moment.** Customer asks about hidden charges. One sentence appears. Hover a teal-underlined figure → document, version, chunk ID. "Every number is traced. Unmarked means unsourced — and unsourced never reaches this screen."
3:00	Their record	Point at the dots. "These four fields are what the co-pilot reads. The credit limit is shown to the agent but never sent to the model — it must not predict a limit."
3:30	Escalation	Credit-terms turn → band turns amber, tier path shows `high`. "A code rule escalated this, not a prompt. Six named rules, served at /api/policy."
4:15	Injection	call-004 has a real attempt. "Halted by an 86-million-parameter classifier for a millionth of a dollar, before any reasoning model saw it."
5:00	Approval	End the call. Before → after diff. "Signing as Priya Nair — taken from the credential, it cannot be typed. This is the only path that writes to a customer record, and no agent can call it."
6:00	**Cost**	Open the evidence strip. "$0.0037 a call, 55× cheaper — from real usage fields, not a model. And 185 of 652 stages cost nothing at all."
7:00	Architecture	One slide: six agents, five models, the code/LLM guardrail split, the four output destinations.
7:30	Roadmap + honesty	"The largest open gap is that grounding validates numbers, not claims like 'we never share your Aadhaar'. That is next."

> **Rehearsal rules**
> Set `VITE_API_KEY=k_agent_priya` to skip the sign-in screen if time is tight. Do not rehearse on the demo's UTC day — the Groq cap is ~7 calls. Keep `SAHAI_MOCK=1` ready as a fallback: every path runs, nothing is billed, and the only visible difference is that costs read zero.

16 · Questions judges will ask

"IS THIS JUST A WRAPPER AROUND AN LLM?"

No — and the ledger proves it. 185 of 652 pipeline stages cost nothing: retrieval is local ONNX embeddings plus BM25, and six of eight guardrails are plain Python over the generated text. The expensive model is invoked on roughly one turn in three, and only when a named code rule says the turn earns it.

"HOW DO YOU STOP IT HALLUCINATING A FEE?"

Three layers. Retrieval withholds source text entirely below a measured similarity floor. The prompt is given only the retrieved chunks. Then `check_grounding` verifies every figure verbatim against the cited chunk — in code, after generation — and blocks the suggestion if one is unaccounted for. The agent sees which figures were traced, underlined in the sentence.

"WHAT IF THE AGENT JUST CLICKS APPROVE ON EVERYTHING?"

They cannot approve a message the guardrail rejected — the server refuses it, and the way past is a rewrite. The rewrite is re-checked against the deterministic rules. The override is recorded in the audit trace under the identity from their credential, which they cannot type.

"WHY NOT USE A FRONTIER MODEL AND ONE GOOD PROMPT?"

It costs 55× more for the same token volume, and that comparison is generous — it prices only the tokens we actually spend, whereas a mega-prompt would carry the whole handbook every turn. More importantly, a single prompt gives you no place to put a deterministic check. You cannot ask a prompt to be un-promptable.

"IS THE COST NUMBER REAL?"

Every row comes from the `usage` field of a real API response, priced by a table in the repo and persisted per decision. Pull `/api/export/trace.csv` and sum the `usd` column — it reproduces the figure exactly.

"WHAT BREAKS FIRST AT SCALE?"

In-memory live-call state. One process is fine; multiple workers need Redis, and that is named in the roadmap. After that, the CRM connector's latency budget — which is why SQLite remains the system of record and the cloud mirrors are asynchronous.

17 · Glossary

Term	Meaning
`NBA`	Next Best Action — the agent that writes the sentence to say
`RRF`	Reciprocal Rank Fusion — merges vector and BM25 rankings without normalising scores
`tier`	Which cost band a decision was routed to: none · tiny · cheap · standard · high · safety · stt
`chunk_id`	Stable identifier for a passage of the handbook, e.g. `objection-cost#0`
`grounded span`	Character offsets of a figure in the suggestion that was traced to a cited chunk
`send_status`	The human-oversight state machine: `pending_agent_approval → approved → sent`, or `rejected`
`enforced_by`	`code` (deterministic, un-promptable) or `llm` (model judgement) — shown in the UI
`principal`	The authenticated identity behind a request: key, display name, role

> **The honest summary**
> Most of this system is not a language model. Retrieval is local. The guardrails that matter are Python. The expensive model is invoked on roughly one turn in three, and only when a code rule says the turn earns it. That is the whole design: **spend intelligence where the stakes are, and nowhere else.** It is why the number is a third of a US cent, and why the safety claims survive contact with a customer trying to break them.